

A Algorithm

Algorithm 1: Overall training pipeline for S2D

Input: Frozen observer model f_θ , training set $\mathcal{S}_{\text{train}}$, null calibration set $\mathcal{S}_{\text{cal}}$ (human-written text only); steering layer ℓ_s ; inverse temperature β ; EMA coefficient ρ ; learning rate η ; epochs E ; batch size B .

Output: Steering vector $\mathbf{v}$, class mean directions $\hat{\boldsymbol{\mu}}_0, \hat{\boldsymbol{\mu}}_1$, and calibrated threshold $\hat{\tau}$.

- 1 **Initialize:** $\mathbf{v} \leftarrow \mathbf{0} \in \mathbb{R}^d$, and uniformly sample $\hat{\boldsymbol{\mu}}_c \in \mathbb{S}^{d-1}$ for $c \in \{0, 1\}$;
 - 2 **for** $e = 1$ **to** E **do**
 - 3 Shuffle $\mathcal{S}_{\text{train}}$ and divide into mini-batches;
 - 4 **for** *each* mini-batch $\mathcal{B} \subset \mathcal{S}_{\text{train}}$ **do**
 - 5 **(1) Forward with steering:** For all $x \in \mathcal{B}$, inject $\mathbf{v}$ at layer ℓ_s :

$$\tilde{h}_{\ell_s, t}(x) \leftarrow h_{\ell_s, t}(x) + \mathbf{v}, \quad \text{for all valid token position } t.$$

Extract the steered representation $f_{\theta, \mathbf{v}}(x) \in \mathbb{S}^{d-1}$ as described in Section 2.2;
 - 6 **(2) Temperature-Scaled Objective:** Compute the batch objective using the softmax form in (2):

$$\mathcal{L}_{\mathcal{B}}(\mathbf{v}) \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(x, y) \in \mathcal{B}} \log q_\beta(y \mid f_{\theta, \mathbf{v}}(x), \hat{\boldsymbol{\mu}}_0, \hat{\boldsymbol{\mu}}_1).$$
 - 7 **(3) Steering Update:** Take a gradient update step: $\mathbf{v} \leftarrow \mathbf{v} + \eta \nabla_{\mathbf{v}} \mathcal{L}_{\mathcal{B}}(\mathbf{v})$;
 - 8 **(4) Mean Directions Update:** For $c \in \{0, 1\}$, compute the batch mean representation $\bar{\mathbf{z}}_c(\mathbf{v})$ and apply EMA:

$$\hat{\boldsymbol{\mu}}_c \leftarrow \frac{(1 - \rho)\hat{\boldsymbol{\mu}}_c + \rho \bar{\mathbf{z}}_c(\mathbf{v})}{\|(1 - \rho)\hat{\boldsymbol{\mu}}_c + \rho \bar{\mathbf{z}}_c(\mathbf{v})\|},$$

where $\bar{\mathbf{z}}_c(\mathbf{v}) = \frac{1}{|\mathcal{B}_c|} \sum_{x \in \mathcal{B}_c} f_{\theta, \mathbf{v}}(x)$, and $\mathcal{B}_c = \{(x, y) \in \mathcal{B} : y = c\}$ denotes the subset of samples in the mini-batch belonging to class c , with $\mathcal{B} = \mathcal{B}_0 \cup \mathcal{B}_1$.
 - 9 **Threshold Calibration:** **for** *each* $x \in \mathcal{S}_{\text{cal}}$ **do**
 - 10 Compute the detection score $\hat{S}(x) = (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0)^\top f_{\theta, \mathbf{v}}(x)$;
 - 11 Set the threshold $\hat{\tau}$ using $\mathcal{S}_{\text{cal}}$ according to the chosen calibration scheme;
 - 12 **return** $(\mathbf{v}, \hat{\boldsymbol{\mu}}_0, \hat{\boldsymbol{\mu}}_1, \hat{\tau})$
-

B Derivation of the Discriminative Posterior Probability

In this section, we derive the class posterior probability $p(y_i \mid f_{\theta, \mathbf{v}}(x_i), \boldsymbol{\mu}_0, \boldsymbol{\mu}_1)$ under the vMF model. In the vMF model, given a text x_i with label $y_i = c \in \{0, 1\}$, its steered representation $f_{\theta, \mathbf{v}}(x_i)$ follows the following distribution on the unit hypersphere $\mathbb{S}^{d-1}$, with likelihood

$$p(f_{\theta, \mathbf{v}}(x_i) \mid y_i = c) = C_d(\kappa) \exp(\kappa \boldsymbol{\mu}_c^\top f_{\theta, \mathbf{v}}(x_i)),$$

where $C_d(\kappa) = \frac{\kappa^{d/2-1}}{(2\pi)^{d/2} I_{d/2-1}(\kappa)}$ is the normalization constant and $\boldsymbol{\mu}_c$ is the class mean direction. We assume a shared concentration parameter κ across classes and a uniform prior $p(y_i = 0) = p(y_i = 1) = \frac{1}{2}$. By Bayes' theorem,

$$p(y_i \mid f_{\theta, \mathbf{v}}(x_i)) = \frac{p(f_{\theta, \mathbf{v}}(x_i) \mid y_i) p(y_i)}{\sum_{c \in \{0, 1\}} p(f_{\theta, \mathbf{v}}(x_i) \mid y_i = c) p(y_i = c)}.$$

Substituting the vMF likelihood and the uniform prior gives

$$p(y_i \mid f_{\theta, \mathbf{v}}(x_i)) = \frac{C_d(\kappa) \exp(\kappa \boldsymbol{\mu}_{y_i}^\top f_{\theta, \mathbf{v}}(x_i)) \cdot \frac{1}{2}}{\sum_{c \in \{0, 1\}} C_d(\kappa) \exp(\kappa \boldsymbol{\mu}_c^\top f_{\theta, \mathbf{v}}(x_i)) \cdot \frac{1}{2}}.$$

Since $C_d(\kappa)$ and the prior $\frac{1}{2}$ are identical across classes, they cancel out, yielding the class posterior

$$p(y_i | f_{\theta, \mathbf{v}}(x_i), \boldsymbol{\mu}_0, \boldsymbol{\mu}_1) = \frac{\exp(\kappa \boldsymbol{\mu}_{y_i}^\top f_{\theta, \mathbf{v}}(x_i))}{\sum_{c \in \{0,1\}} \exp(\kappa \boldsymbol{\mu}_c^\top f_{\theta, \mathbf{v}}(x_i))}.$$

Thus, the shared-concentration vMF model induces a softmax posterior with scale κ . In Phase I, we retain this functional form while replacing the distributional concentration κ with a tunable inverse temperature β , which yields the optimization objective in (2).

C Alternative Threshold Calibration via the Youden Index

In standard benchmarking settings, where a balanced trade-off between the true positive rate (TPR) and false positive rate (FPR) is desired, one may select the threshold by maximizing the Youden index [78] on a calibration set. Let the calibration sample be given by $\mathcal{S}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^{n_2}$, where $y_i \in \{0, 1\}$ indicates whether x_i is LLM-generated ($y_i = 1$) or human-written ($y_i = 0$). For a threshold τ , the empirical TPR and FPR are defined as

$$\widehat{\text{TPR}}(\tau) := \frac{1}{n_+} \sum_{i: y_i=1} \mathbb{1}(\widehat{\mathcal{S}}(x_i) \geq \tau), \quad \widehat{\text{FPR}}(\tau) := \frac{1}{n_-} \sum_{i: y_i=0} \mathbb{1}(\widehat{\mathcal{S}}(x_i) \geq \tau),$$

where $n_+ = \sum_{i=1}^{n_2} \mathbb{1}(y_i = 1)$ and $n_- = \sum_{i=1}^{n_2} \mathbb{1}(y_i = 0)$. The Youden-based threshold is then given by

$$\widehat{\tau}_Y := \arg \max_{\tau} (\widehat{\text{TPR}}(\tau) + 1 - \widehat{\text{FPR}}(\tau)).$$

Equivalently, this criterion maximizes $\widehat{\text{TPR}}(\tau) - \widehat{\text{FPR}}(\tau)$, thereby favoring thresholds that achieve high detection power while maintaining a low FPR.

Empirical comparison of Type-I error control and power. To further understand the practical behavior of different calibration strategies, we compare the threshold defined in (5), denoted by $\widehat{\tau}_\alpha$, with the Youden-based threshold $\widehat{\tau}_Y$ in terms of both their realized Type-I error and empirical power.

We use *meta-llama/Llama-3.1-8B* as the observer model. Both the training set and the calibration set are drawn from a mixed dataset comprising outputs from four LLMs together with the corresponding human-written texts, while evaluation is conducted on four test sets generated by *ChatGPT-3.5-Turbo*, *Llama-2-70B*, *Google-PaLM*, and *Claude-Instant*, respectively. For each calibration strategy, the threshold is computed using the same mixed calibration set, and we report the realized Type-I error and empirical detection power. Since $\widehat{\tau}_Y$ does not depend on a target level α , it yields a single operating point once the calibration set is fixed, whereas $\widehat{\tau}_\alpha$ explicitly adapts to the desired Type-I error level. We focus on the representative operating regime $\alpha = 1\%$.

Table 6: Realized type-I error (%) and empirical detection power (%) under different calibration strategies at target level $\alpha = 1\%$.

Method	ChatGPT-3.5-Turbo		Llama-2-70B		Google-PaLM		Claude-Instant	
	Type-I Error	Power	Type-I Error	Power	Type-I Error	Power	Type-I Error	Power
$\widehat{\tau}_Y$	0.20	99.70	0.00	99.50	0.20	85.70	0.20	99.80
$\widehat{\tau}_\alpha$	0.90	100.00	0.80	99.90	0.80	92.10	1.30	99.90

As shown in Table 6, the threshold $\widehat{\tau}_\alpha$ achieves realized Type-I errors close to the target level of 1% across all test sets, with FPRs ranging from 0.80% to 1.30%, which is consistent with the finite-sample guarantee in Theorem 3.1. At the same time, it maintains strong empirical power, reaching at least 99.90% on three generators and substantially improving performance on *Google-PaLM*. By contrast, the Youden-based threshold $\widehat{\tau}_Y$ yields a single fixed operating point and does not explicitly target a prescribed Type-I error level. In our experiments, it behaves conservatively, with realized FPRs between 0.00% and 0.20%, all well below the target level of 1%. Although this still gives strong detection performance on some generators, it also reduces power by imposing a stricter threshold than necessary; this is most evident on *Google-PaLM*, where the TPR decreases from 92.10% under

$\hat{\tau}_\alpha$ to 85.70% under $\hat{\tau}_Y$. Overall, these results show that while $\hat{\tau}_Y$ is suitable for a fixed balanced operating point, $\hat{\tau}_\alpha$ is more flexible with respect to the desired false positive budget and can yield better power by avoiding unnecessarily conservative thresholding.

D Experiment Details

This section outlines the experimental setup and evaluation criteria.

Evaluation Metrics. To comprehensively evaluate the discriminative capability of each detector under different security requirements, we adopt the Area Under the Receiver Operating Characteristic curve (AUROC) and the True Positive Rate (TPR) at fixed False Positive Rate (FPR) thresholds, namely $\text{TPR@FPR}=1\%$ and $\text{TPR@FPR}=0.01\%$. Since the empirical ROC curve is defined over discrete test samples, we use linear interpolation to report TPR at the specified FPR levels. In particular, with 1,000 human-written test samples, the smallest nonzero empirical FPR is 0.1%, so $\text{TPR@0.01\%}$ is an interpolated low-FPR summary rather than a directly observed operating point. Appendix F.2 provides a large-sample diagnostic of this interpolation.

Setup. To ensure comparability, we standardize backbone models across all detectors. We use *meta-llama/Llama-3.1-8B* as the base model for scoring, rewriting, and representation extraction. For training-free methods, the same open-source LLM is used as a surrogate to compute statistics (e.g., Likelihood, LRR, FDGPT). In Binoculars, *meta-llama/Llama-3.1-8B* serves as the observer and *meta-llama/Llama-3.1-8B-Instruct* as the performer. For training-based approaches (e.g., RoBERTa), we report both models fine-tuned on our data and publicly available checkpoints⁷. For rewrite-based methods, rewrites are generated by *meta-llama/Llama-3.1-8B-Instruct* with 4 samples, following [23].

For the experiments in Sections 4.1 and 4.2 (excluding the ID/OOD experiments), the detectors are trained on a mixed dataset composed of outputs from four LLMs and evaluated under diverse settings. We consistently use *meta-llama/Llama-3.1-8B* as the observer model, and *meta-llama/Llama-3.1-8B-Instruct* for methods requiring rewriting. The rewriting process incurs a substantial computational cost, amounting to approximately 300 GPU hours on NVIDIA A100 80GB GPUs. For experiments comparing with watermarking methods, we employ *Qwen/Qwen2.5-7B* as the base model to generate watermarked text using three different watermarking schemes.

For our method, unless otherwise specified, we adopt a fixed configuration across all experiments. The steering vector is injected at layer 11 through the residual connection. Representations are extracted from the last 8 transformer layers, where we average the final 25% tokens in each layer to obtain the representation. Training is conducted using AdamW [79] with a learning rate of 1×10^{-3} , batch size 8, and 10 epochs. The EMA decay for centroid updates is set to 0.9, and the optimization inverse temperature is set to $\beta = 2.5$. All hyperparameters are kept fixed across datasets and models without extensive tuning.

E Empirical Motivation for Representation Modeling

Our modeling choices for f_θ are supported by the following two empirical properties of hidden representations observed across models and domains:

1. **Norm concentration.** By pooling hidden states from the final 20% of tokens across the last $N = 8$ layers, we find that the resulting L_2 norms are tightly concentrated for *EleutherAI/GPT-J-6B*, *Qwen/Qwen2.5-7B*, and *meta-llama/Llama-3.1-8B* (Figure 5). This supports a hyperspherical approximation, where semantic information is primarily encoded in direction rather than magnitude.
2. **Directional unimodality.** On the unit sphere, the normalized representations exhibit a unimodal directional structure within each class (human or LLM), as evidenced by the distribution of $\hat{\mu}^\top f_\theta(x)$ relative to the estimated class mean directions $\hat{\mu}$ (Figure 6). Moreover, the estimated concentration parameters $\hat{\kappa}$ are stable across domains and remain similar between the human and LLM classes.

⁷<https://huggingface.co/openai-community/roberta-large-openai-detector>